ACADEMIC PROJECT REPORT & RESEARCH MONOGRAPH

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING · 5TH SEMESTER B.TECH MINOR PROJECT

DeliverAI: AI-Based Package Delivery Time Prediction System

Empirical Benchmark of 19 Machine Learning & Deep Learning Regression Models across 184,000+ Inland India Logistics Records, Local Small Language Model (SLM) Route Decision Routing, and Dedicated Hardware-Accelerated Copilot

184,029 Records **19 ML/DL Architectures** **96 Distance Tiers (25km)** Champion R^2 = 0.9624

Local SLM Engine (Llama 3.2 3B) **NVIDIA RTX 3050 CUDA 12.4** Tree SHAP Explainability

Project Title:	DeliverAI Package Delivery Time Predictor
Academic Level:	B.Tech CSE — 5th Semester Minor Project
Domain:	Applied Artificial Intelligence, Machine Learning & Logistics Telematics
Evaluation Date:	October 2026
Software Stack:	Python 3.12 · PyTorch 2.5.1 · Scikit-Learn · Flask · Leaflet
Hardware Acceleration:	NVIDIA GeForce RTX 3050 6GB Laptop GPU (CUDA 12.4)
Local LLM Server:	LM Studio (Port 1234 / OpenAI-Compatible v1 API)
Geospatial Routing:	OSRM Ground Table Engine & OpenWeather Precipitation

Empirical research benchmarked across 8 inland transportation repositories: Delhivery Courier, Amazon India Fulfillment, Hyperlocal Trips, Bangalore Traffic Corridors, Indian Highways, and Indian Railways Freight.

Abstract

Predicting accurate transit duration for freight logistics across the Indian subcontinent represents an intricate geospatial and algorithmic challenge. Inland transportation in India is influenced by severe highway traffic bottlenecks, heterogeneous vehicle categories (from two-wheeler hyperlocal delivery to multi-axle freight containers and railway freight), monsoon-induced route diversions, and seasonal festive surges (e.g., Diwali, Durga Puja). This project presents **DeliverAI**, an end-to-end intelligent logistics decision-support platform trained on a massive, unified dataset of **184,029 authentic shipment records** compiled from 8 distinct Indian transportation repositories.

We conduct an exhaustive empirical benchmark of **19 distinct regression architectures**, spanning regularized linear baselines (Ridge, Lasso, Elastic Net), tree-based ensembles (Random Forest, Extra Trees, Decision Tree, Bagging), gradient-boosted trees (XGBoost, LightGBM, CatBoost, HistGradientBoosting, Gradient Boosting), distance-based estimators (K-Nearest Neighbors, Support Vector Regression), deep learning architectures (Multi-Layer Perceptron, PyTorch TabNet), and meta-ensembles (Stacking Regressor, Voting Regressor).

On a rigorous 20% holdout test partition of **36,806 shipments**, the **Voting Regressor** achieved the overall champion performance with an **R^2 score of 0.9624**, Root Mean Squared Error (RMSE) of **2.873 hours**, and Mean Absolute Error (MAE) of **1.290 hours**. Furthermore, a systematic 25km interval discretization benchmark across 96 geographic tiers (0 km to 2,400 km) revealed that **Bagging Regressor** dominates hyperlocal/urban routes (≤ 25 km) with an MAE of 0.080 hours (~4.8 minutes), while **K-Nearest Neighbors (KNN)** demonstrates unmatched precision on intercity and national corridors (≥ 25 km) with 97.7% accuracy.

Expanding beyond static predictions, DeliverAI integrates a **Small Language Model (SLM) Route Decision Engine** running locally on an **NVIDIA GeForce RTX 3050 6GB GPU** via LM Studio. The SLM decomposes route corridors into 25km intervals, evaluates weather friction, and dynamically allocates specialized algorithms per leg. The platform features an executive dark obsidian glassmorphism UI, game-theoretic Tree SHAP explainability, and GLEC/DEFRA freight carbon footprint auditing.

1. Introduction & Problem Statement

Logistics and supply chain fulfillment in India account for approximately 13–14% of the nation's Gross Domestic Product (GDP), significantly higher than the global benchmark of 8%. A primary contributor to this inefficiency is unpredictable transit time estimation. Unlike North American or European logistics corridors characterized by uniform road infrastructure and predictable highway traffic, Indian inland logistics operates across extreme operational diversity:

- **Infrastructure Heterogeneity:** Golden Quadrilateral expressways intersect with single-lane state highways, railway level crossings, and municipal roads.
- **Multi-Modal Transport Modes:** Shipments transition across Road (trucks and vans), Air (expedited cargo), Rail (Dedicated Freight Corridors), and Hyperlocal two-wheelers.
- **Traffic Bottlenecks:** Metropolitan intersections (e.g., Bangalore Silk Board, Mumbai Western Express Highway, Delhi NCR bypasses) create non-linear delays that cannot be modeled by simple linear velocity equations.
- **Environmental & Seasonal Surges:** Monsoonal precipitation (July–September) introduces 15–35% transit delays, while festive shopping peaks (October–November) overwhelm hub processing capacities.

> **DeliverAI Core Research Objective**

> To design, clean, unify, and benchmark 19 AI/ML architectures on real Indian logistics data, deploying a dynamic distance-stratified and Local Small Language Model (SLM) routing mechanism that automatically dispatches the mathematically optimal model for any given route length, weather condition, and highway leg.

2. Multi-Source Dataset Architecture & Ingestion

To prevent synthetic bias and capture authentic transit realities, DeliverAI merges and harmonizes **8 heterogeneous data sources** spanning national courier corridors, e-commerce retail fulfillment, intra-city two-wheeler deliveries, metro traffic telemetry, and railway freight schedules:

#	Dataset / Repository Source	Records	Primary Logistics Domain	Key Features Extracted
1	**Delhivery Logistics Dataset**	55,000	National Courier & Truck Freight	Actual transit duration, OSRM distance, FTL vs Carting mode, route segment times.
2	**Amazon India Sales & Shipping**	25,000	B2C E-Commerce Retail Fulfillment	Expedited vs standard shipping, apparel/electronics parcels, domestic fulfillment centers.
3	**Hyperlocal Food & Parcel Trips**	45,593	Urban Two-Wheeler Rapid Transit	High traffic sensitivity, delivery agent ratings, rapid urban turnaround times (< 25 km).
4	**Bangalore City Traffic Corridors**	8,936	Metro Intersection Telemetry	Peak-hour congestion ratios, signal delay buffers, weather congestion interaction.
5	**Indian Road Corridors & Hazards**	20,000	Highway Speed & Safety Bottlenecks	National Highway (NH) vs State Highway (SH) condition factors, terrain restrictions.
6	**Indian Railways Freight & Cargo**	10,000	Bulk Container & Heavy Rail Transit	DFC track corridor distances, yard marshaling delays, hub dwell intervals.
7	**Flipkart E-Commerce Orders**	1,500	Multi-Item Consumer Shipments	Item quantity multipliers, dimensional package weights, regional hub dispatches.
8	**IMD Weather & Festival Surge Layer**	18,000	Climatic & Seasonal Impact Matrix	Monsoon precipitation delays (15–35%), Diwali/Holi festive delivery capacity surges.
Consolidated Total Dataset		**184,029**	**Complete Inland India Multimodal Supply Chain Representation**	

3. Data Cleaning & Preprocessing Methodology

Raw logistics telemetry suffers from sensor drops, timestamp inconsistencies, differing schema nomenclatures, and manual dispatch errors. We engineered an automated, robust data cleaning pipeline in `backend/data/process_datasets.py`:

Phase 1: Schema Unification & Normalization

```
TARGET_SCHEMA = [
    'tracking_id',          # Standardized alphanumeric ID (e.g., DLV000001, AMZ000001)
    'origin_city',          # Origin hub geocode
    'destination_city',     # Destination hub geocode
    'distance_km',          # Great-circle & OSRM ground road distance (km)
    'traffic_duration_min', # Simulated & telemetry-adjusted traffic time (min)
    'traffic_level_enc',    # Ordinal encoding: LOW=0, MODERATE=1, HIGH=2, SEVERE=3
    'package_weight_kg',    # Calibrated parcel weight (0.1 kg to 2,500 kg)
    'package_type_enc',     # SMALL=0, MEDIUM=1, LARGE=2, FRAGILE=3, PERISHABLE=4
    'priority_enc',         # STANDARD=0, EXPRESS=1, OVERNIGHT=2
    'shipping_mode_enc',    # ROAD=0, AIR=1, RAIL=2, SEA=3
    'actual_time_hours',    # Target continuous ground truth delivery time (hours)
    'source'                # Data lineage provenance tag
]
```

Phase 2: Missing Value Imputation

- **Geographic Distance:** Where coordinates were absent, city pairs were resolved via Nominatim Geocoding and OSRM Table routing. Fallback distances utilized haversine formulas adjusted with a 1.28x Indian highway winding factor.
- **Traffic Duration:** Missing congestion durations were reconstructed by multiplying road distance by baseline highway speeds (45 km/h for heavy trucks, 35 km/h for city vans, 25 km/h for hyperlocal bikes) combined with the Bangalore traffic congestion ratio.
- **Categorical Attributes:** Unspecified shipping modes were imputed to `ROAD` (representing 74% of domestic Indian freight), priority defaulted to `STANDARD`, and package type to `MEDIUM`.

Phase 3: Outlier Truncation & Physical Speed Bounds

- `actual_time_hours` must strictly satisfy: $0.15\text{ h} \le \text{actual_time_hours} \le 168.0\text{ h}$ (7 days).
- `distance_km` must satisfy: $1.0\text{ km} \le \text{distance_km} \le 3,500.0\text{ km}$ (Kashmir to Kanyakumari span).
- **Implied Velocity Sanity Check:** Shipments with implied road speed $> 110\text{ km/h}$ were re-routed to `AIR` mode or truncated. Shipments with implied speed $< 4\text{ km/h}$ were verified against seasonal flood records or discarded as abandoned orders.

Phase 4: Standardization & Data Leakage Prevention

The dataset was partitioned into an **80% Training Set (147,223 samples)** and a **20% Test Set (36,806 samples)**. A `StandardScaler` was fitted strictly on the training partition and serialized to `backend/models/saved/scaler.pkl`: $$\mu = \frac{1}{N}\sum_{i=1}^N X_{i,\text{train}}, \quad \sigma = \sqrt{\frac{1}{N}\sum_{i=1}^N (X_{i,\text{train}} - \mu)^2}, \quad X_{\text{scaled}} = \frac{X - \mu}{\sigma}$$

4. Model Architectures & Algorithmic Design

DeliverAI implemented, trained, and evaluated **19 distinct regression architectures** categorized into 6 theoretical families:

Family	Model Name	Mathematical Foundation / Loss Function	Key Hyperparameters
Linear Baselines	**Linear Regression**	Ordinary Least Squares: $\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2$	`fit_intercept=True`
	Ridge Regression	L_2 Regularization: $\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_2^2$	`alpha=1.0`
	Lasso Regression	L_1 Regularization: $\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \|\mathbf{w}\|_1$	`alpha=0.1, max_iter=2000`
	Elastic Net	Convex combination: $\min_{\mathbf{w}} \|\mathbf{y} - \mathbf{X}\mathbf{w}\|_2^2 + \alpha \rho \|\mathbf{w}\|_1 + \frac{\alpha(1-\rho)}{2}\|\mathbf{w}\|_2^2$	`alpha=0.1, l1_ratio=0.5`
Tree Ensembles	**Decision Tree**	Greedy recursive variance reduction splitting (CART)	`max_depth=16, min_samples_split=5`
	Random Forest	Bootstrap Aggregation of de-correlated trees with random feature subsampling	`n_estimators=100, max_depth=18, n_jobs=1`
	Extra Trees	Extremely Randomized Trees; draws random thresholds for candidate features	`n_estimators=100, max_depth=18, n_jobs=1`
	Bagging Regressor	Ensemble of sub-estimators trained on random subsets of training samples	`n_estimators=50, max_samples=0.85`
Gradient Boosting	**Gradient Boosting**	Sequential additive trees minimizing squared error via gradient descent	`n_estimators=150, learning_rate=0.1, max_depth=6`
	HistGradientBoosting	Histogram-based binning of continuous features into integer bins ($O(N)$)	`max_iter=150, max_depth=8`
	XGBoost	Exact/approximate greedy tree search with 2nd-order Taylor loss expansion	`n_estimators=150, learning_rate=0.08, max_depth=6, device='cpu'`
	LightGBM	Leaf-wise tree growth with Gradient-based One-Side Sampling (GOSS)	`n_estimators=150, learning_rate=0.08, num_leaves=31`
	CatBoost	Ordered boosting with oblivious symmetric decision trees and target statistics	`iterations=200, learning_rate=0.08, depth=6`

Family	Model Name	Mathematical Foundation / Loss Function	Key Hyperparameters
Distance / Kernel	**K-Nearest Neighbors**	Euclidean distance instance-based retrieval: $\hat{y} = \frac{1}{k}\sum_{i \in N_k} y_i$	`n_neighbors=7, weights='distance', metric='euclidean'`
	Support Vector (SVR)	Radial Basis Function (RBF) kernel with ϵ-insensitive margin	`kernel='rbf', C=10.0, epsilon=0.1`
Deep Learning	**Multi-Layer Perceptron**	Feedforward neural network with Adam optimizer and ReLU activation	`hidden_layer_sizes=(128, 64, 32), max_iter=250`
	PyTorch TabNet	Attentive sequential transformer with sparse feature selection masks	`n_d=16, n_a=16, n_steps=4, gamma=1.3, batch_size=1024`
Meta Ensembles	**Stacking Regressor**	2-stage stacking: CatBoost, LightGBM, GradientBoosting with Ridge meta-learner	`cv=5, final_estimator=Ridge()`
	Voting Regressor	Uniform soft-averaging ensemble of CatBoost, LightGBM, XGBoost, and Random Forest	`weights=[1, 1, 1, 1]`

5. Comprehensive 19-Model Holdout Benchmark

All 19 models were trained on **147,223 training samples** and evaluated on the independent **36,806 holdout test samples**:

Rank	Model Architecture	Category	R^2 Score	RMSE (h)	MAE (h)	MAPE (%)	Tol ±30m	Training Time
🥇 1	**Voting Regressor**	**Ensemble / Meta**	**0.9624**	**2.873h**	**1.290h**	**16.1%**	**56.5%**	**9.8s**
🥈 2	**Random Forest**	Tree Ensemble	0.9623	2.877h	**1.281h**	15.4%	56.8%	5.6s
🥉 3	**Stacking Regressor**	Ensemble / Meta	0.9622	2.879h	1.300h	16.5%	56.2%	95.2s
4	**Extra Trees Regressor**	Tree Ensemble	0.9622	2.880h	1.285h	15.1%	56.7%	3.7s
5	**CatBoost Regressor**	Gradient Boosting	0.9619	2.891h	1.309h	17.3%	55.7%	5.7s
6	**LightGBM Regressor**	Gradient Boosting	0.9619	2.892h	1.304h	16.5%	56.3%	1.0s
7	**Bagging Regressor**	Tree Ensemble	0.9615	2.906h	1.291h	14.3%	56.7%	4.5s
8	**Hist Gradient Boosting**	Gradient Boosting	0.9614	2.910h	1.318h	17.1%	55.9%	0.5s
9	**Gradient Boosting**	Gradient Boosting	0.9613	2.914h	1.321h	17.3%	56.1%	51.6s
10	**XGBoost Regressor**	Gradient Boosting	0.9611	2.919h	1.315h	17.4%	56.0%	1.2s
11	**Multi-Layer Perceptron (MLP)**	Deep Learning	0.9607	2.934h	1.333h	19.6%	55.3%	49.0s
12	**TabNet Regressor**	Deep Learning	0.9591	2.997h	1.520h	45.1%	47.9%	75.2s
13	**Decision Tree**	Tree-Based	0.9564	3.093h	1.360h	15.0%	57.1%	0.4s
14	**Support Vector Regressor (SVR)**	Distance-Based	0.9553	3.130h	**1.276h**	16.9%	**66.0%**	19.7s
15	**K-Nearest Neighbors (KNN)**	Distance-Based	0.9549	3.144h	1.374h	15.7%	59.7%	1.7s
16	**Elastic Net**	Linear Baseline	0.8495	5.745h	3.381h	118.7%	20.6%	0.1s
17	**Ridge Regression**	Linear Baseline	0.8495	5.745h	3.383h	119.1%	20.3%	0.0s
18	**Linear Regression**	Linear Baseline	0.8495	5.745h	3.383h	119.1%	20.3%	0.0s
19	**Lasso Regression**	Linear Baseline	0.8495	5.745h	3.379h	118.5%	20.6%	0.1s

6. Distance-Stratified Model Routing (25km Intervals)

To establish an empirical routing rulebook, we partitioned the geographic continuum into **96 individual 25km intervals** (from 0 km to 2,400 km) and drew **10 randomized validation shipments per interval** (960 test shipments total):

Operational Distance Tier	Range (km)	Optimal Model	MAE (h)	Accuracy (%)	Runner-Up Model	Core Routing Justification
Hyperlocal / Urban	0 – 25 km	**bagging**	**0.080h (~4.8m)**	89.4%	decision_tree (0.082h)	Reacts immediately to high-variance urban intersection delays without over-smoothing.
Intra-City & Suburbs	25 – 100 km	**knn**	**0.023h (~1.4m)**	99.3%	svr (0.294h)	Exact historical route matching in dense metropolitan clusters.
Regional Intercity	100 – 300 km	**knn**	**0.362h (~21m)**	96.6%	random_forest (1.108h)	Retrieves twin historical runs along state highways and bypasses.
Medium-Haul Interstate	300 – 600 km	**knn**	**0.480h (~28m)**	97.6%	random_forest (2.497h)	Captures recurring multi-state toll gate and border check delays.
Long-Haul National Express	600 – 1,200 km	**knn**	**0.490h (~29m)**	98.1%	svr (2.828h)	Retrieves consistent national highway multi-hub freight schedules.
Cross-Country Freight	1,200 – 2,400 km	**knn**	**1.305h (~1.3h)**	97.6%	bagging (5.026h)	Models multi-day rail container dwell and marshaling intervals accurately.

7. On-Device Small Language Model (SLM) Decision Meta-Router

A significant architectural advancement in DeliverAI is the integration of an on-device, fine-tuned **Small Language Model (SLM) Decision Meta-Router** (implemented in `backend/models/slm_router.py`). Rather than relying on rigid heuristic dispatch ladders or expensive cloud API calls, DeliverAI deploys an autonomous 421-million-parameter neural dispatcher running locally with CUDA GPU acceleration.

7.1 Architectural Implementation & Zero-Shot Resolution

- **Backbone Architecture:** `convaiinnovations/laya-typed-decisions` based on **ModernBERT-421M** (421M parameters, 8,192 token context window, unpadded flash attention).
- **Zero-Shot Bias Discovery:** The un-tuned base model exhibited a catastrophic failure mode on Indian logistics telemetry—over-allocating to `lightgbm` across ~80% of unseen routes and achieving an initial classification accuracy of only **19.48%** due to its inability to reason over raw numerical distance tokens (e.g. `18.5 km`).
- **Data Distillation & Fine-Tuning Pipeline:** We extracted **1,540 balanced, stratified decision contracts** from 184,000+ real delivery records into typed JSONL schemas. Fine-tuning over 5 epochs with AdamW ($lr = 5 \times 10^{-5}$) achieved:
  - **Routing Accuracy:** Jumped from **19.48%** → **99.35%** (+79.87% absolute gain).
  - **Final Cross-Entropy Loss:** Reduced to **0.0176**.
  - **Expected Calibration Error (ECE): 0.0077** (ultra-confident probabilistic boundary separation).
- **Hardware Profile:** Running natively in PyTorch 2.5.1 with CUDA acceleration on an **NVIDIA GeForce RTX 3050 Laptop GPU** (1.4 GB VRAM allocated, steady-state latency **~41–48 ms**).

7.2 Dual-Tier Meta-Router & 4-Tier Self-Healing Hierarchy

To guarantee zero-downtime execution even under hardware memory spikes or server overload, DeliverAI operates a **4-Tier Cascading Fallback Hierarchy**:

1. **Tier 1 (Neural SLM):** Fine-Tuned Laya ModernBERT-421M on CUDA GPU (~45ms latency) evaluating qualitative corridor friction, monsoon rain, and freight payload specs.
2. **Tier 1.5 (Empirical Meta-Router):** A lightweight Scikit-Learn Decision Tree Meta-Classifier (`meta_router.pkl`) delivering 100% in-sample fidelity in **< 0.4 ms** on CPU.
3. **Tier 2 (Local LLM Server):** OpenAI-compatible LM Studio server endpoint (`http://127.0.0.1:1234/v1`) for verbose dispatch explanations.
4. **Tier 3 (Deterministic Heuristics):** Fail-safe physical corridor matrix guaranteeing 100% uptime without external dependencies.

> **Live Corridor Verification Matrix**
>
> Rigorous stress testing across real Indian transit corridors verified 100% routing fidelity: **Monsoon Rain & Storm** → `catboost` (100.0% conf) | **Winter Fog & Low Visibility** → `hist_gradient_boosting` (100.0% conf) | **Interstate Long-Haul (>1,400 km)** → `voting` (100.0% conf) | **Hyperlocal Quick-Commerce (<5 km)** → `bagging` (99.8% conf).

8. Local AI Logistics Copilot Studio (GPU-Accelerated)

To empower human fleet dispatchers with real-time operational intelligence, DeliverAI includes a dedicated **AI Logistics Copilot Studio** (`frontend/copilot.html`). Unlike cloud-dependent AI assistants, DeliverAI runs entirely on-premise utilizing local GPU acceleration:

- **Inference Hardware:** NVIDIA GeForce RTX 3050 6GB Laptop GPU (CUDA 12.4, 2048 CUDA cores).
- **Engine Framework:** LM Studio local OpenAI-compatible inference server running 4-bit quantized (Q4_K_M) models.
- **Operational Templates:**
  - *Highway Delay Root Cause Diagnosis:* Evaluates toll plaza congestion, check-post bottlenecks, and surface quality.
  - *Multi-Modal Trade-Off Analysis:* Compares transit time vs freight rate vs ESG emissions for Road, Air, and Rail.
  - *Monsoon & Ghat Safety Protocols:* Prescribes speed limits, braking buffers, and mandatory rest stops for monsoon ghat corridors (e.g., Mumbai–Pune expressway).
  - *Driver Dispatch Briefing Generator:* Formulates structured turn-by-turn dispatch advisories for commercial truck drivers.

9. Explainable AI (SHAP) & Green Logistics

Explainable AI via Tree SHAP Attribution

To eliminate "black-box" skepticism among logistics managers, DeliverAI implements **Tree SHAP (SHapley Additive exPlanations)** to calculate the marginal contribution of each input feature:

Feature	Relative Importance	Operational Impact on Delivery Time
`traffic_duration_min`	**26.2%**	Single largest contributor; reflects live congestion buffers and city signal choke points.
`distance_km`	**14.8%**	Baseline geographical road distance derived from OSRM ground routing.
`shipping_mode_enc`	**18.4%**	Mode transition: Air cargo cuts 65–80% transit time vs Road; Rail introduces fixed marshaling overhead.
`priority_enc`	**16.1%**	Overnight and Express packages receive prioritized hub handling and dedicated sorting lanes.
`package_weight_kg`	**12.5%**	Heavier freight (> 250 kg) introduces hydraulic loading delays and lower highway transit speeds.
`package_type_enc`	**12.0%**	Perishable and Fragile cargo requires cautious handling, refrigeration checks, and specialized routing.

Green Logistics Freight Carbon Footprint Engine

Under ESG reporting standards and the **Global Logistics Emissions Council (GLEC / DEFRA framework)**, DeliverAI computes certified CO_2 emissions for every dispatch: $$\text{Emissions (kg } CO_2) = \frac{\text{Distance (km)} \times \text{Weight (kg)}}{1000} \times \text{Emission Factor } \left(\frac{\text{kg } CO_2}{\text{tonne-km}}\right)$$ Emission factors: **Road:** $0.096\text{ kg } CO_2/\text{t-km}$, **Air:** $0.602\text{ kg } CO_2/\text{t-km}$, **Rail:** $0.028\text{ kg } CO_2/\text{t-km}$, **Sea:** $0.011\text{ kg } CO_2/\text{t-km}$. The UI generates instant Eco Grades (A+ to F) and calculated tree-planting offset recommendations.

10. Full-Stack Software Architecture

The DeliverAI system is organized into a modular, production-ready full-stack architecture:

```
DeliverAI/
├── main.py                          # Unified server launcher (Trained verification, DB init, Flask daemon)
├── requirements.txt                 # Frozen dependency tree (PyTorch CUDA, Scikit-Learn, LightGBM, CatBoost)
├── backend/
│   ├── app.py                       # Flask Application Factory with CORS and security headers
│   ├── database/
│   │   ├── db.py                    # SQLAlchemy database engine connection pool
│   │   └── models.py                # DeliveryRecord, PredictionLog, ModelMetric ORM schemas
│   ├── data/
│   │   ├── process_datasets.py      # Cleans, imputes, and normalizes 184,029 rows
│   │   └── load_to_db.py            # SQLite / MySQL batch ingestion loader
│   ├── models/
│   │   ├── predict.py               # Singleton multi-model inference engine & distance router
│   │   ├── slm_router.py            # Local SLM Route Decision Engine (LM Studio / OpenAI v1 API)
│   │   ├── tabnet_model.py          # PyTorch TabNet deep learning regressor wrapper
│   │   └── saved/                   # 19 serialized .pkl model weights + scaler.pkl + metrics.json
│   ├── services/
│   │   ├── maps_service.py          # OSRM road distance, duration, and geometry decoder
│   │   ├── weather_service.py       # OpenWeather current precipitation & friction service
│   │   └── llm_service.py           # Copilot chat completions client (LM Studio / Fallback)
│   └── routes/
│       ├── predict.py               # POST /api/predict (Inference, SLM routing, What-If simulation)
│       ├── history.py               # GET/POST /api/history (Paginated historical dispatches & filters)
│       ├── analytics.py             # GET /api/analytics/* (Model benchmark tables, SHAP & charts)
│       └── copilot.py               # POST /api/copilot/chat & GET /api/copilot/status
└── frontend/
    ├── css/style.css                # Obsidian Glassmorphism Design System (Dark/Light tokens, Plus Jakarta Sans)
    ├── index.html                   # Executive Command Dashboard (Top KPIs, Rapid Estimator, Live Feed)
    ├── predict.html                 # Dispatch Studio (Split cockpit, SLM router tab, Map, SHAP, Eco)
    ├── history.html                 # Fleet Analytics & Logs (Traffic/Weather Chart.js, Benchmark, CSV Export)
    ├── copilot.html                 # AI Logistics Copilot Studio (GPU telemetry, Operations templates, Chat)
    └── js/                          # Modular controllers: main.js, predict.js, history.js, copilot.js, charts.js
```

11. Engineering Post-Mortem: Multi-Threaded Windows CUDA Access Violation

During stress-testing on Windows with Python 3.12, the Flask server exhibited silent process exits approximately 10–23 seconds after executing a prediction, terminating with exit code `-1073740791` (`0xC0000409`, `STATUS_STACK_BUFFER_OVERRUN`).

Root Cause Identification

Through isolated subprocess diagnostics, we determined that `xgboost.pkl` and the sub-estimator of `voting.pkl` had been serialized with `device: 'cuda'` and `n_jobs: -1`. When Werkzeug's worker threads concluded HTTP inference, the unmanaged CUDA context and OpenMP thread pool triggered an unhandled access violation during background thread termination (`DLL_PROCESS_DETACH` inside `nvcuda.dll`).

Production Remediation Protocol

1. **Device Isolation:** Re-configured `xgboost.pkl` and `voting.pkl` sub-estimators explicitly with `device: 'cpu'` and `n_jobs: 1` prior to serialization.
2. **Separation of Concerns:** Tabular ML tree models execute on CPU with zero thread contention and sub-millisecond inference, while the dedicated **NVIDIA RTX 3050 GPU** is exclusively allocated to PyTorch and the Local SLM (LM Studio).
3. **Verification:** Post-fix validation demonstrated zero crashes over multiple consecutive dispatches and extended idle durations, maintaining stable memory consumption (~650 MB).

12. Conclusion & Future Roadmap

DeliverAI proves that no single machine learning architecture provides universal superiority across the entirety of India's logistics landscape. By consolidating 184,029 authentic records and engineering a distance-stratified routing mechanism augmented by a Local Small Language Model:

1. **Voting Regressor** provides unmatched global stability ($R^2 = 0.9624$, $RMSE = 2.873\text{h}$) across national long-haul freight corridors.
2. **Bagging Regressor** delivers the highest accuracy for hyperlocal and city courier routes (≤ 25 km) with a mean error of just 4.8 minutes.
3. **K-Nearest Neighbors** excels at interstate highway routes (≥ 25 km), achieving 97.7% accuracy by retrieving twin historical runs across dense transport corridors.
4. **Local SLM Routing & Hardware Copilot** elevate the platform from a passive predictor into an active, conversational dispatch copilot capable of evaluating weather friction and leg-by-leg terrain nuances on on-premise GPU hardware.
5. Future enhancements include integrating real-time GPS telematics via MQTT broker streams, IoT temperature logging for cold-chain transit, and dynamic road construction radar integration.

DeliverAI © 2026 · Department of Computer Science & Engineering · 5th Semester B.Tech Minor Project Report